

Inductive data

Robert Y. Lewis

CS 0220 2022

April 27, 2022

Overview

1 Why does induction work?

2 Other inductive sets

3 Inductive proofs

4 Closing

Induction on $\mathbb{N}$

We introduced induction as a technique to prove things about natural numbers.

It makes some intuitive sense. But let's examine things more carefully.

Defining $\mathbb{N}$

What are the natural numbers?

- 1 0 is a natural number.
- 2 For any natural number k , $k + 1$ is a natural number. $\text{successor}(k)$
- 3 successor is injective.
- 4 For every k , $\text{successor}(k) \neq 0$.
- 5 Every natural number is either 0 or the successor of another natural number.

Are there any sets that satisfy properties 1-5 that *don't* look like $\mathbb{N}$?

Defining $\mathbb{N}$

Let's try again.

- 1 0 is a natural number.
- 2 For any natural number k , $k + 1$ is a natural number. $\text{successor}(k)$
- 3 successor is injective.
- 4 For every k , $\text{successor}(k) \neq 0$.
- 5 Every natural number can be represented as a (finite) directed tree, where each node is either
 - labeled 0, and has no children; or
 - labeled successor , and has one child.

Condition 5 is equivalent to the principle of induction.

Again, succinctly

We define $\mathbb{N}$ to be an *inductive set* with *constructors*

- $0 : \mathbb{N}$
- successor: $\mathbb{N} \rightarrow \mathbb{N}$.

An inductive set is defined by giving a list of constructors that are assumed to satisfy properties 3-5.

See also: *inductive types* or *algebraic data types* in some programming languages.

And, recursion

Let A be any set, $a \in A$, and $g : \mathbb{N} \times A \rightarrow A$. There exists a unique function $f : \mathbb{N} \rightarrow A$ satisfying the two clauses:

- $f(0) = a$
- $f(k + 1) = g(k, f(k))$

“Exists” and “unique.” In other words: “to define a function with domain $\mathbb{N}$, we can describe its behavior on the two constructors.”

Sounds a lot like induction. And the tree property.

Inductive lists

Let A be a set. The set $L(A)$ of lists of elements of A is an inductive set with constructors

- $\text{nil} : L(A)$
- $\text{cons} : A \times L(A) \rightarrow L(A)$

“To create a list, either create the empty list, or take a list and tack on one more value.”

Induction on lists

- $\text{nil} : L(A)$
- $\text{cons} : A \times L(A) \rightarrow L(A)$

Tree property?

Induction principle?

To show that $P(l)$ holds for every list $l \in L(A)$, show:

- $P(\text{nil})$
- For every $a \in A$ and $l \in L(A)$, $P(l) \rightarrow P(\text{cons}(a, l))$

Inductive graphs?

Let A be a set of “labels.” The set $G(A)$ of A -labeled graphs is an inductive set with constructors

- $single : A \rightarrow G(A)$
- $edge : G(A) \times A \times A \rightarrow G(A)$
- $vertex : G(A) \times A \rightarrow G(A)$

Yes? No? Anything fishy here?

We can write down some “syntax trees” that don’t look like good graphs. And we can write down multiple syntax trees that look like the same graph.

Inductive graphs :(

Let A be a set of “labels.” The set $G(A)$ of A -labeled graphs is an inductive set with constructors

- $single : A \rightarrow G(A)$
- $edge : G(A) \times A \times A \rightarrow G(A)$
- $vertex : G(A) \times A \rightarrow G(A)$

Inductive formulas

The set F of formulas of propositional logic is an inductive set with constructors

- $letter : \mathbb{N} \rightarrow F$
- $not : F \rightarrow F$
- $and : F \times F \rightarrow F$
- $or : F \times F \rightarrow F$
- $implies : F \times F \rightarrow F$
- $iff : F \times F \rightarrow F$

Principle of induction? To prove $P(\varphi)$ holds for every prop formula φ , it suffices to show:

- $P(letter(i))$ for every i (“ P holds of every propositional letter”)
- $P(\varphi) \rightarrow P(not(\varphi))$
- $P(\varphi_1) \wedge P(\varphi_2) \rightarrow P(and(\varphi_1, \varphi_2))$
- $P(\varphi_1) \wedge P(\varphi_2) \rightarrow P(or(\varphi_1, \varphi_2))$
- ...

Inductive formulas

Recursion on formulas, in words:

To define a function $f : F \rightarrow A$, it suffices to describe the behavior of F on each constructor of F .

Example: evaluation $E(\varphi)$ under a propositional assignment $v : \mathbb{N} \rightarrow \{T, F\}$.

- $E(\text{letter}(i)) = v(i)$
- $E(\text{not}(\varphi)) = \text{NOT}(E(\varphi))$
- $E(\text{and}(\varphi_1, \varphi_2)) = \text{AND}(E(\varphi_1), E(\varphi_2))$

Challenge: phrase this like we phrased recursion on $\mathbb{N}$.

Proofs as data

We have a technique for figuring out if a propositional formula is *valid*: write the truth table, see if all columns are T.

This is more of a “process” than an “object.” Intuition: if you write down an argument like this, the only way I can check it is by doing it myself and comparing.

Other ways?

Proofs as data: introduction rules

How can I prove $\varphi_1 \wedge \varphi_2$? Prove φ_1 and then prove φ_2 .

How can I prove $\varphi_1 \vee \varphi_2$? Prove φ_1 . Alternatively, prove φ_2 .

This sounds sort of inductive. Constructors?

- $and_intro : proof(\varphi_1) \times proof(\varphi_2) \rightarrow proof(\varphi_1 \wedge \varphi_2)$
- $or_intro_left : proof(\varphi_1) \rightarrow proof(\varphi_1 \vee \varphi_2)$
- $or_intro_right : proof(\varphi_2) \rightarrow proof(\varphi_1 \vee \varphi_2)$

Proofs as data: elimination rules

What can I do with a proof of $\varphi_1 \wedge \varphi_2$? Prove φ_1 . Alternatively, prove φ_2 .

- $and_elim_left : proof(\varphi_1 \wedge \varphi_2) \rightarrow proof(\varphi_1)$
- $and_elim_right : proof(\varphi_1 \wedge \varphi_2) \rightarrow proof(\varphi_2)$

What can I do with a proof of $\varphi_1 \vee \varphi_2$? Case split...tricky.

Need to analyze implication first, which also muddies the picture a bit.

For another time

Can't dive into the details now. But we can make things more or less work.

Induction on proofs?? “I can only construct proofs of valid formulas.”

Recursion on proofs?? Given a proof, reconstruct the formula it proves—proof checking!

Proofs are directed trees!

Final thoughts

We've seen a lot of topics this semester. Remember why we've done this:

- Vocabulary. Use the languages of logic, combinatorics, probability, ... as a shared, precise vocabulary for discussing problems.
- Abstraction. A lot of the problems we've studied will show up in different contexts, in and out of computer science. Remember our abstract solutions and adapt them to reality.
- Team problem solving. CS is collaborative, and hopefully you've gotten practice solving problems with a team.